



LV18 DFS/BFS 경로 탐색, 최소 여행비용

<https://youtu.be/WQvVYwoIM18?feature=shared>

<https://youtu.be/hedl1fXTMjk?feature=shared>

<https://youtu.be/ccLAT4xZXQI?feature=shared>

모든 경로 출력하기 수업노트

핵심 개념

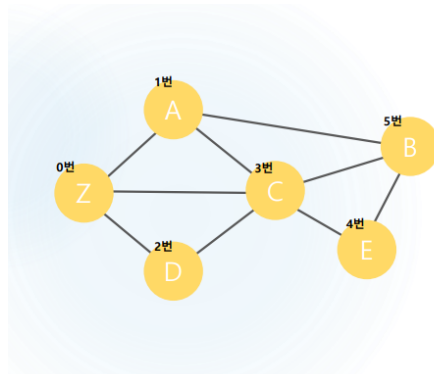
이번시간에는 DFS/BFS 알고리즘을 사용해서 경로를 출력하거나 최소비용을 찾아보도록 하자

다음과 같은 그래프가 존재 할때

path 배열을 운영하여, 특정 노드에 도착하는 모든 경로를 출력해보자.

path 배열은 level 전달 인자와 함께 사용하여야 한다.

DFS 모든 경로 탐색 소스코드



핵심 아이디어

- path 배열: 현재까지의 경로를 저장
- level 인자: 현재 깊이와 path 배열의 인덱스 역할
- 목표 노드 도달: 특정 노드(5번)에 도달하면 경로 출력
- 백트래킹: 탐색 후 visited 배열과 path 배열 원상복구

DFS 구현 코드

```
#include <iostream>
#include <queue>
```

```

char value[10] = "ZADCEB";
int visited[10] = { };
int map[6][6] =
{
    {0, 1, 1, 1, 0, 0},
    {1, 0, 0, 1, 0, 1},
    {1, 0, 0, 1, 0, 0},
    {1, 1, 1, 0, 1, 1},
    {0, 0, 0, 1, 0, 1},
    {0, 1, 0, 1, 1, 0}
};

int cnt = 0;
char path[10] = { };

bool flag = false;
void dfs(int now, int level)
{
    if (now == 5)          // 목표 노드(E)에 도달
    {
        flag = true;
        cnt++;
        std::cout << path << std::endl; // 경로 출력
        return;
    }

    for (size_t i = 0; i < 6; i++)
    {
        if (map[now][i] == 1 && visited[i] == 0) // 연결되고 미방문
        {
            visited[i] = 1;          // 방문 처리
            path[level+1] = value[i]; // 경로에 추가
            dfs(i, level + 1);        // 재귀 호출
            path[level + 1] = 0;      // 백트래킹: 경로 제거
            visited[i] = 0;          // 백트래킹: 방문 해제
        }
    }
}

int main()
{
    visited[0] = 1; // 시작점 방문 처리
    path[0] = value[0]; // 시작점을 경로에 추가
    dfs(0, 0);

    return 0;
}

```

핵심 포인트:

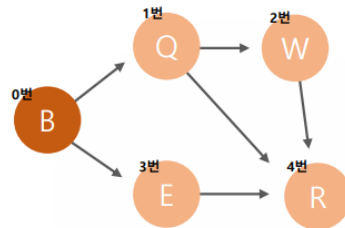
1. **경로 저장:** `path[level+1] = value[i]` 로 현재 경로 기록
2. **백트래킹:** 재귀 호출 후 `path[level+1] = 0` , `visited[i] = 0` 로 원상복구
3. **목표 도달:** `now == 5` 일 때 경로 출력하고 반환
4. **모든 경로:** 백트래킹으로 인해 모든 가능한 경로 탐색

BFS로 모든 노드 탐색하는 것과 경로 탐색의 차이

BFS로 모든 노드를 탐색하는 것은 used전역배열을 이용하여 각 노드에 방문했는지 중복체크를 하면 된다.

BFS로 경로를 탐색하기 위해서는 Queue에서 각 노드마다 자기 노드를 어떻게 방문했는지 경로(path)를 노드마다 저장해주면 된다.

BFS 모든 경로 탐색 소스코드



핵심 아이디어

- **Node 구조체:** data, level, path[10] 포함
- **각 노드마다 개별 경로:** 큐의 각 노드가 자신만의 경로 정보 보유
- **경로 복사:** 새로운 노드 생성 시 현재 경로를 복사하고 추가

BFS 구현 코드

```
#include <iostream>
#include <queue>

struct Node
{
    int data;
    int level;
    int path[10] = {};
};

int map[5][5] =
{
    {0, 1, 0, 1, 0},
    {0, 0, 1, 0, 1},
    {0, 0, 0, 0, 1},
    {0, 0, 0, 0, 1},
    {0, 0, 0, 0, 0}
};

std::queue<Node> queue;
char value[10] = "BQMER";

int main()
{
    queue.push({ 0, 0, {1,0,0,0,0,0,0,0,0,0} }); // 시작노드: B(0)

    while (!queue.empty())
    {
        Node now = queue.front();
        if (value[now.data] == 'R') // 목표 노드 R에 도달
        {
            // ...
        }
    }
}
```

```

//std::cout << now.path[0] << now.path[1] << now.path[2] << now.path[3] << now.path
for (size_t i = 0; i < 5; i++)
{
    if (now.path[i] == 1)
    {
        std::cout << value[i];
    }
}
std::cout << std::endl;
}

for (int i = 0; i < 5; i++)
{
    if (map[now.data][i] == 0) // 연결되지 않으면 건너뛰기
        continue;
    if (now.path[i] == 1) // 이미 방문한 노드면 건너뛰기
        continue;

    Node next = now; // 현재 노드 정보 복사
    next.data = i; // 다음 노드로 설정
    next.level = now.level + 1; // 레벨 증가
    next.path[i] = 1; // 경로에 추가

    queue.push(next);
}
queue.pop();
}

return 0;
}

```

핵심 포인트:

1. 개별 경로 관리: 각 Node가 자신만의 `path[10]` 배열 보유
2. 경로 복사: `Node next = now` 로 현재 경로 정보 복사
3. 경로 추가: `next.path[i] = 1` 로 새로운 노드를 경로에 추가
4. 사이클 방지: `now.path[i] == 1` 체크로 이미 방문한 노드 제외

DFS vs BFS 경로 탐색 비교

구분	DFS	BFS
경로 저장 방식	전역 path 배열 + 백트래킹	각 노드별 개별 path 배열
메모리 사용	적음 (재귀 스택)	많음 (모든 노드가 경로 복사)
구현 복잡도	백트래킹 필요	구조체 설계 필요
탐색 순서	깊이 우선	너비 우선
최단 경로	보장 안됨	첫 번째 발견이 최단

주요 차이점 정리

DFS 방식

- 공유 배열: 하나의 `path[]` 배열을 모든 재귀 호출이 공유
- 백트래킹: 재귀 호출 후 `visited[]` 와 `path[]` 원상복구 필수
- 메모리 효율: 스택 공간만 사용

BFS 방식

- 개별 배열: 각 큐 노드가 독립적인 `path[]` 배열 소유
- 경로 복사: 새 노드 생성시 현재 경로 정보 복사
- 메모리 사용량: 큐에 저장된 모든 노드가 경로 정보 보관

선택 기준:

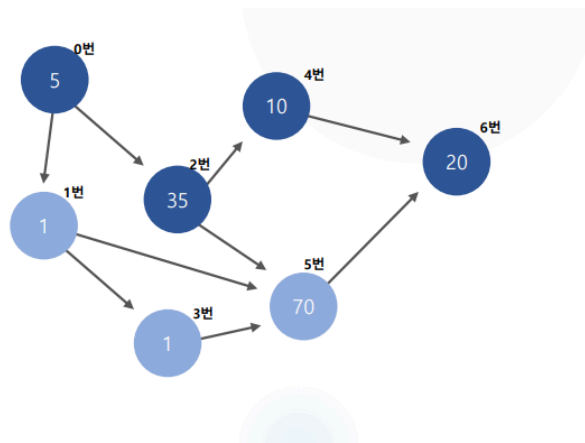
- 모든 경로 필요 → DFS
- 최단 경로 우선 → BFS

최소 여행 비용 찾기

핵심 개념

각 노드에는 평균 여행 비용이 적혀 있다.

0번에서 6번 노드까지 갈 때 최소 여행 비용을 구하는 문제를 DFS/BFS로 구현 해보자.



DFS 최소 여행 비용 찾기

핵심 아이디어

- DFS로 모든 경로를 탐색한다.
- 각 노드의 값을 재귀함수에 `sum` 전달인자에 누적합을 구한다.
- 6번 노드에 도착 했을 때, 최소가 되는 값을 전역변수를 통해서 기록한다.

DFS 구현 코드

```
#include <iostream>
#include <queue>

int map[7][7] =
{
    {0, 1, 1, 0, 0, 0, 0},
    {0, 0, 0, 1, 0, 1, 0},
    {0, 0, 0, 0, 1, 0, 0},
    {0, 0, 0, 0, 0, 1, 0},
    {0, 0, 0, 0, 0, 0, 1},
    {0, 0, 0, 0, 0, 0, 1},
    {0, 0, 0, 0, 0, 0, 0}
};
```

```

int value[10] = { 5, 1, 35, 1, 10, 70, 20 };
int visited[7] = { 1, };
int minValue = 987654321/*21e8*/;
int path[7] = { value[0],};

void dfs(int now, int sum, int level)
{
    if (now == 6)
    {
        if (minValue > sum)
        {
            minValue = sum;
        }
        return;
    }

    for (size_t i = 0; i < 7; i++)
    {
        if (map[now][i] == 0) continue;
        if (visited[i] == 1) continue;

        visited[i] = 1;
        dfs(i, sum + value[i], level + 1);
        visited[i] = 0;
    }
}

int main()
{
    dfs(0, value[0], 0);
    std::cout << minValue << std::endl;

    return 0;
}

```

핵심 포인트:

1. **sum 누적:** `dfs(i, sum + value[i], level + 1)` 로 비용 누적
2. **최솟값 갱신:** `if (minValue > sum)` 조건으로 최솟값 업데이트
3. **백트래킹:** `visited[i] = 0` 으로 방문 상태 복원
4. **초기값:** `minValue = 987654321` 로 충분히 큰 값 설정

BFS 최소 여행 비용 찾기

핵심 아이디어

- 각 노드마다 개별 정보 관리: `num`, `sum`, `used[]` 배열
- 목표 도달시 최솟값 비교: 6번 노드 도달 시 `minValue` 갱신
- 경로별 독립적 탐색: 각 큐 노드가 독립적인 방문 이력 보유

BFS 구현 코드

```

#include <iostream>
#include <queue>

int map[7][7] =

```

```

{
    {0, 1, 1, 0, 0, 0, 0},
    {0, 0, 0, 1, 0, 1, 0},
    {0, 0, 0, 0, 1, 0, 0},
    {0, 0, 0, 0, 0, 1, 0},
    {0, 0, 0, 0, 0, 0, 1},
    {0, 0, 0, 0, 0, 0, 1},
    {0, 0, 0, 0, 0, 0, 0}
};

int value[10] = { 5, 1, 35, 1, 10, 70, 20 };
int minValue = 987654321/*21e8*/;
int path[7] = { value[0],};

struct Node
{
    int num;
    int sum;
    int used[7];
};

std::queue<Node> queue;

int main()
{
    queue.push({ 0, value[0], {1, 0, 0, 0, 0, 0, 0} });

    while (!queue.empty())
    {
        Node now = queue.front();

        if (now.num == 6 && minValue > now.sum)
        {
            minValue = now.sum;
        }

        for (size_t i = 0; i < 7; i++)
        {
            if (map[now.num][i] == 0) continue;
            if (now.used[i] == 1) continue;

            Node next = now;
            next.num = i;
            next.sum += value[i];
            next.used[i] = 1;

            queue.push(next);
        }

        queue.pop();
    }

    return 0;
}

```

핵심 포인트:

1. 구조체 설계: `Node` 에 `num` , `sum` , `used[7]` 모두 포함

2. 개별 경로 관리: 각 노드가 독립적인 `used[]` 배열 보유
3. 비용 누적: `next.sum += value[i]` 로 비용 추가
4. 상태 복사: `Node next = now` 로 현재 상태 복사

가중 그래프

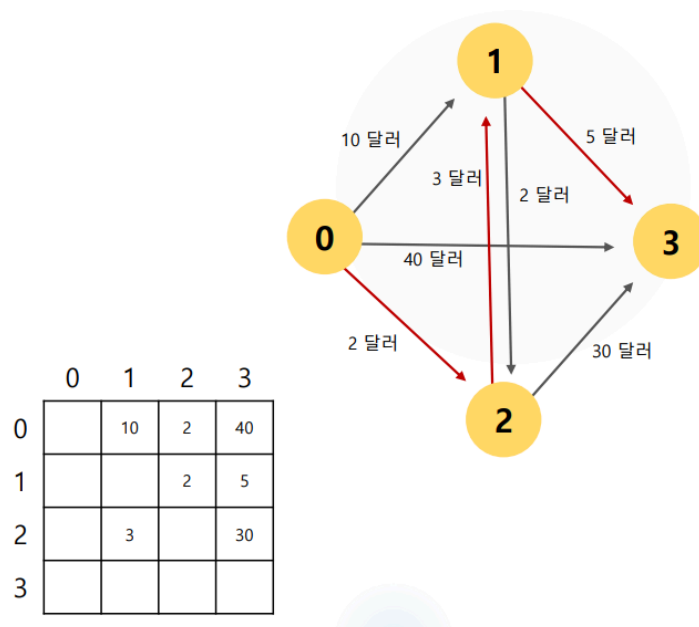
기본 개념

노드에 가중치 값, 즉 비용을 붙인 그래프를 뜻한다.

인접행렬에 0 또는 1 대신, 가중치를 쓰면된다.

가중 그래프 표현 방법

인접 행렬 표현



기존 그래프 DFS/BFS에서는 `sum(경로의합)`을 구할때 가중치 값을 이용하여 구하면 된다.

DFS vs BFS 비교 (최소 비용 탐색)

구분	DFS	BFS
탐색 방식	깊이 우선 (재귀)	너비 우선 (큐)
메모리 사용	적음 (스택)	많음 (모든 경로 저장)
백트래킹	필요 (<code>visited[i] = 0</code>)	불필요 (개별 배열)
최적해 보장	모든 경로 탐색 후	모든 경로 탐색 후
구현 복잡도	간단 (재귀)	복잡 (구조체 설계)

🎯 "강의는 많은데, 내 실력은 왜 그대로일까?"

혼자서 공부하다 보면

이런 생각 들지 않으셨나요?

- 강의는 다 듣고도 직접 코드는 못 짜겠고,

- 복습할 땐 어디서부터 다시 시작해야 할지 막막하고,
- 질문하려 해도 물어볼 사람이 없고,
- 유튜브 영상도 정답만 보고 따라 치는 느낌...

그렇다면 지금이 바로

“나만을 위한 코칭”이 필요한 순간입니다.

 **당신도 할 수 있습니다.**

지금 멤버십을 넘어, 코칭에 도전해보세요.

수많은 수강생들이 **암암코딩 코칭**으로 넥슨, 크래프톤, NC 등 입사에 성공했습니다.

 **프리미엄 코칭 안내 바로가기**

 또는 카톡 오픈채팅: [암암코딩 상담방](#)

지금도 코딩을 ‘따라 치기만’ 하고 계신가요?

이젠 **혼자 설계하고, 스스로 코딩하는 법**을 배워야 할 때입니다.

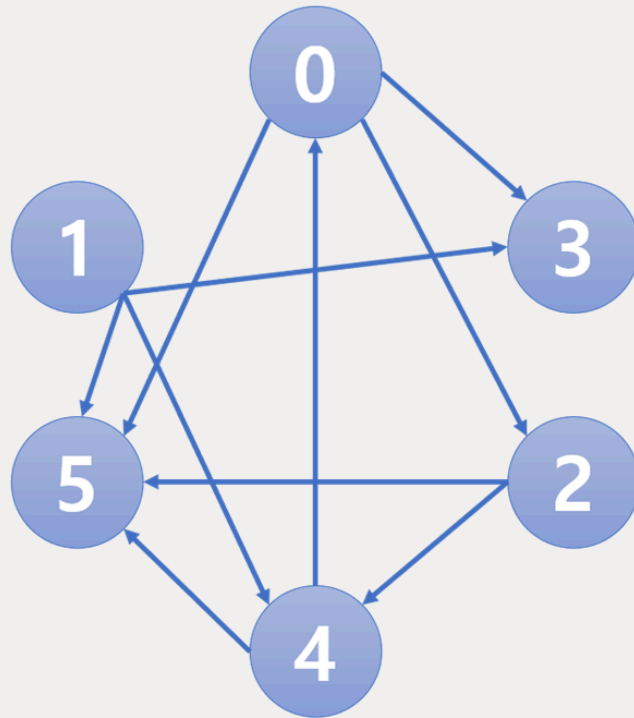
암암코딩이 옆에서 함께하겠습니다. 💪

▼ oopy:hide



문제 1번

인접 행렬 그래프를 깊이 우선 탐색법으로 탐색 해봅시다.



[그래프]

그래프를 나타내는 인접 행렬을 하드코딩 하세요.

출발지점의 노드 값을 입력 받으세요.

입력 받는 노드 부터 탐색을 시작할때,

DFS로 노드들을 방문할 때마다 노드의 값을 출력해 주세요.

한 노드에 여러 노드로 갈 수 있다면,

숫자가 작은 노드부터 탐색 해주세요.

Ex) 0에서 세개의 노드로 갈 수 있는데, 작은 숫자부터 차례로 2, 3, 5 순으로 탐색이 진행되어야 합니다.

[하드코딩 용 테이블]

	0	1	2	3	4	5
0			1	1		1
1				1	1	1
2					1	1
3						
4	1					1
5						

입력 예제

0

출력 결과

0 2 4 5 3



문제 2번

각 노드들을 연결하는 선에는 가중치가 저장되어 있습니다.

가중치가 저장되어 있는 그래프를 인접행렬로 하드 코딩하고 출발 지점을 입력 받으세요.

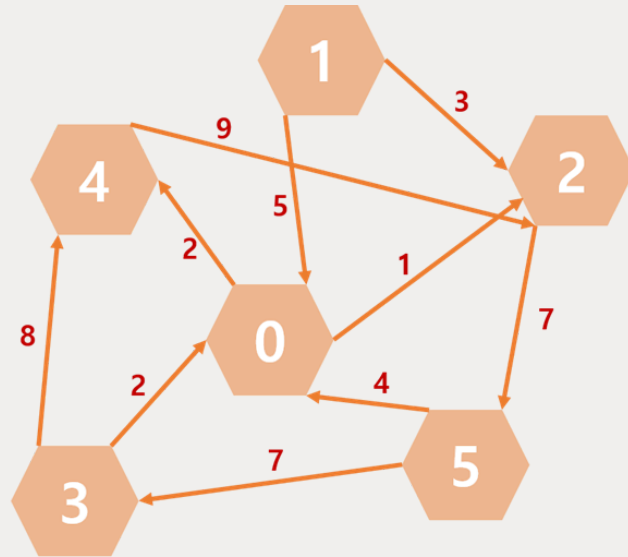
출발 지점부터 깊이 우선 탐색법으로 탐색했을때 노드 번호와 가중치 경로의 합을 출력해 주세요.

한번 방문한 노드는 다시 방문할 수 없습니다. 출발지점의 가중치 시작 값은 0 입니다.

한 노드에 여러 노드가 붙어있다면, 숫자가 작은 순서대로 탐색을 시도 합니다.

Ex) 아래 예제는 0은 2와 4로 이동할 수 있습니다. 2와 4중에 작은 숫자인 2부터 탐색을 시작합니다.

[가중치 그래프]



[가중치 그래프 테이블]

	0	1	2	3	4	5
0			1		2	
1	5		3			
2						7
3	2				8	
4			9			
5	4			7		

입력 예제

0

출력 결과

0 0

2 1

5 8

3 15

4 23



문제 3번

트리 자료구조에서 너비 우선 탐색법으로 각 노드를 탐색해주세요.

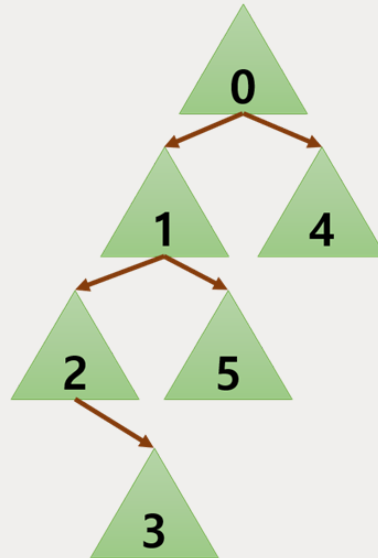
시작 지점 부터, 노드에 방문할 때마다 값을 출력 해주세요.

출발지점은 입력으로 주어 집니다.

한번 방문 했던 노드는 방문할 수 없습니다.

트리 자료구조는 인접행렬로 하드코딩 해주세요.

[트리]



[하드코딩용 테이블]

	0	1	2	3	4	5
0		1			1	
1			1			1
2				1		
3						
4						
5						

입력 예제

0

출력 결과

0 1 4 2 5 3



문제 4번

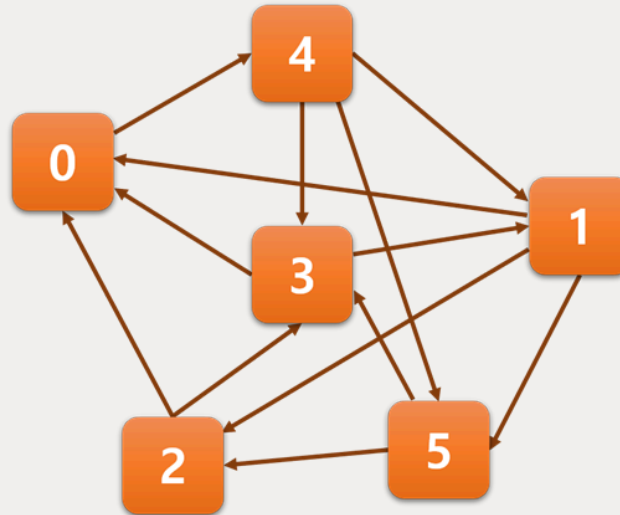
아래에 그래프가 놓여져 있습니다.

입력 받은 출발 지점에서 너비 우선 탐색법으로 그래프를 탐색해주세요.

한번 방문했던 노드는 다시 방문할 수 없습니다.

시작 지점부터 BFS가 끝날때까지 방문한 노드를 출력해주세요.

[그래프]



[하드코딩용 테이블]

	0	1	2	3	4	5
0					1	
1	1		1			1
2	1			1		
3	1	1				
4		1		1		1
5			1	1		

입력 예제

0

출력 결과

0

4

1

3

5

2

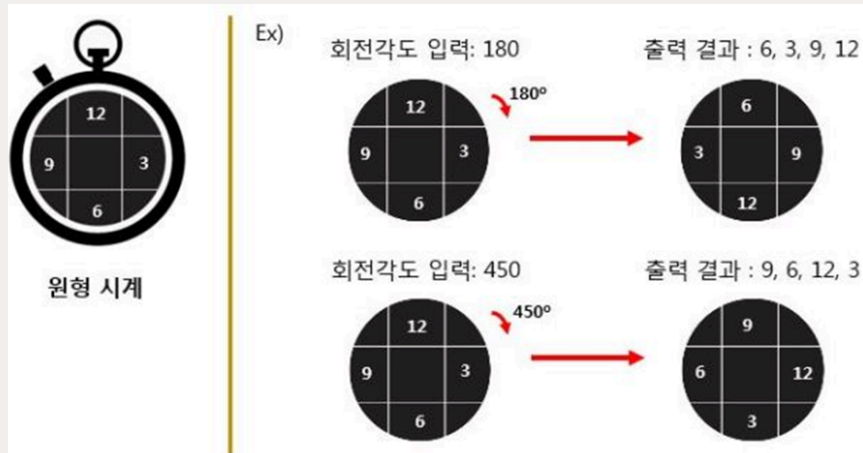


문제 1번

원형시계가 있습니다.

몇도 돌릴것인지 90도의 배수로 입력 받아주세요.

그리고 시계를 시계방향으로 돌리고, 그 결과를 출력 하세요.



입력 예제

180

출력 결과

6 3 9 12



문제 2번

축구 결승전에서 승부차기를 하게 되었습니다.

oxox 이면 첫번째/세번째 사람이 공을 넣은 것입니다.

승부차기 할 사람수를 입력 받고, 가능한 경우수를 출력 하세요.

(재귀호출로 구현해주세요)

ex)

[입력]

3

[출력]

ooo

ooX

OXO

OXX

XOO

XOX

XXO

XXX

입력 예제

4

출력 결과

oooo

ooOX

ooXO

ooXX

OXOO

OXOX

OXXO

OXXX

XOOO

XOOX

XOXO

XOXX

XXOO

XXOX

XXXO

XXXX



문제 3번

아래 배열을 하드 코딩 하주세요.

1	5	4	2	-5	-7
---	---	---	---	----	----

n을 입력받고, n번째 큰 숫자를 출력 하주세요.

만약 3을 입력받았다면, 세 번째로 큰 숫자는 $5 > 4 > 2$ 이기 때문에 2를 출력하면 됩니다.

입력 예제

3

출력 결과

2



문제 4번

0과 1로된 문자열 3개 입력 받으세요. (최대 30글자)

2진수라고 생각 했을 때, 가장 큰 숫자를 출력 해주세요.

ex)

[입력]

1101011

11110

1101110

[출력]

1101110

입력 예제

1001011

11110

1101110

출력 결과

1101110



문제 5번

후보들을 입력 받으세요.

그리고 입력 한 후보 중 몇명을 선출할지 뽑을 인원을 입력 받으세요. (숫자 1개 입력)

중복을 허용하여 뽑을 수 있는 후보의 경우를 출력 해주세요.

(Backtracking을 이용해서 풀어주세요)

Ex)

후보입력 : AGFT ↵

뽑을 인원 : 2 ↵

<출력>

AA

AG

AF

AT

GA

GG

GF

GT

...

TF

TT

입력 예제

AGFT

2

출력 결과

AA

AG

AF

AT

GA

GG

GF

GT

FA

FG

FF

FT

TA

TG

TF

TT

💡 문제 6번



A ~ Z 까지 대입을 해봐서 암호를 알아내려고 합니다.

아래의 순서대로 대입을 해보려합니다.

AAAA

AAAB

AAAC

...

ZZZZ

자전거 암호 n개가 존재할때, 몇회만에 각 암호를 해제 할 수 있는지 출력하세요.

ex)

[입력]

3 (자전거 암호 개수)

AAAC (암호 입력)

ATKC (암호 입력)

ZBAB (암호 입력)

[출력]

3

13107

440078

입력 예제

3

AAAC

ATKC

ZBAB

출력 결과

3

13107

440078



문제 7번

다섯 숫자를 아래 배열에 입력 받으세요.

--	--	--	--	--

입력받은 숫자들 중 몇 개를 뽑아서

더했을 때 값이 10~20 되는 조합이 총 몇 가지 인지 Counting 해주세요

(Backtracking으로 구현 해 주세요)

ex)

[입력]

1 3 4 7 9

[출력]

3+4+7

3+4+9

4+7

7+9

1+3+4+7

...

18

[힌트]

이진트리 형태로 생각해보면 됩니다.

입력이 1 3 5 7 9 라면

첫번째 Level에서는 숫자 1을 쓸지 or 말지

두번째 Level에서는 숫자 3을 쓸지 or 말지

세번째 Level에서는 숫자 5를 쓸지 or 말지

네번째 Level에서는 숫자 7를 쓸지 or 말지

다섯번째 Level에서는 숫자 9를 쓸지 or 말지

입력 예제

5 4 3 9 1

출력 결과

16



문제 8번

B, I, A, H 슈퍼영웅들 중 출동할 사람을 순서대로 뽑아야 합니다.

척척박사님은 자신의 이름이 다섯글자이기에 영웅B를 시작으로 n번째 사람을 선택합니다.

출동하는 영웅들의 순서를 출력 하세요.

(큐를 이용하지 않고, For문 or While문을 활용해서 풀어주세요)

만약 5를 입력받았다면,

항상 다섯번째 사람을 먼저 출동시키면 됩니다.

B	I	A	H
			

B	I	A	H
①	②	③	④
⑤			

→ 첫번째 출동자는 배동현

B	I	A	H
	①	②	③
	④	⑤	

→ 두번째 출동자는 캡틴A

B	I	A	H
			①
			②
			③
			④
			⑤

→ 세번째 출동자는 헐크오

출동순서 결과: B A H I

입력 예제

5

출력 결과

B A H I



문제 :

정수 배열을 항상 증가 순으로 유지하는 SortedArray 클래스를 작성하려고 한다.

아래의 main() 함수가 작동할 만큼만 SortedArray 클래스를 작성하고 +와 = 연산자도 작성하라.

```

class SortedArray{
    int size; // 현재 배열의 크기
    int *p; // 정수 배열에 대한 포인터
    void sort(); // 정수 배열을 오름차순으로 정렬
public:
    SortedArray(); // p는 NULL로 size는 0으로 초기화
    SortedArray(SortedArray& src); // 복사 생성자
    SortedArray(int p[], int size); // 생성자. 정수 배열과 크기를 전달받음
    ~SortedArray(); // 소멸자
    SortedArray operator+ (SortedArray& op2); // 현재 배열에 op2 배열 추가
    SortedArray& operator= (const SortedArray& op2); // 현재 배열에 op2 배열 복사
    void show(); // 배열의 원소 출력
};

```

```

int main() {
    int n[] = { 2, 20, 6 };
    int m[] = { 10, 7, 8, 30 };
    SortedArray a(n, 3), b(m, 4), c;

    c = a + b; // +, = 연산자 작성 필요
    // + 연산자가 SortedArray 객체를 리턴하므로 복사 생성자 필요

    a.show();
    b.show();
    c.show();
}

```

실행결과 :

```

배열 출력 : 2 6 20
배열 출력 : 7 8 10 30
배열 출력 : 2 6 7 8 10 20 30

-----
Process exited after 6.9 seconds with return value 0
계속하려면 아무 키나 누르십시오 . . .

```

목적 및 힌트 :

연산자 중복, 복사 생성자의 종합 응용

+ 연산자는 SortedArray 객체를 리턴하므로 복사 생성자가 반드시 필요하다.

a=b; 연산에서 = 연산자는 객체a의 배열 메모리를 모두 delete 시키고 객체 b의 크기만큼 다시 할당받은 후 객체 b의 배열 내용을 복사하도록 작성되어야 한다.

고급 알고리즘 준비문제



문제 2번

올림픽 100m 달리기에서 **n명이 출전** 했습니다.



이중 1, 2, 3 등을 뽑아야하는데 나올 수 있는 경우의 수는 몇 가지인지 출력 해주세요.

입력 예제

4

출력 결과

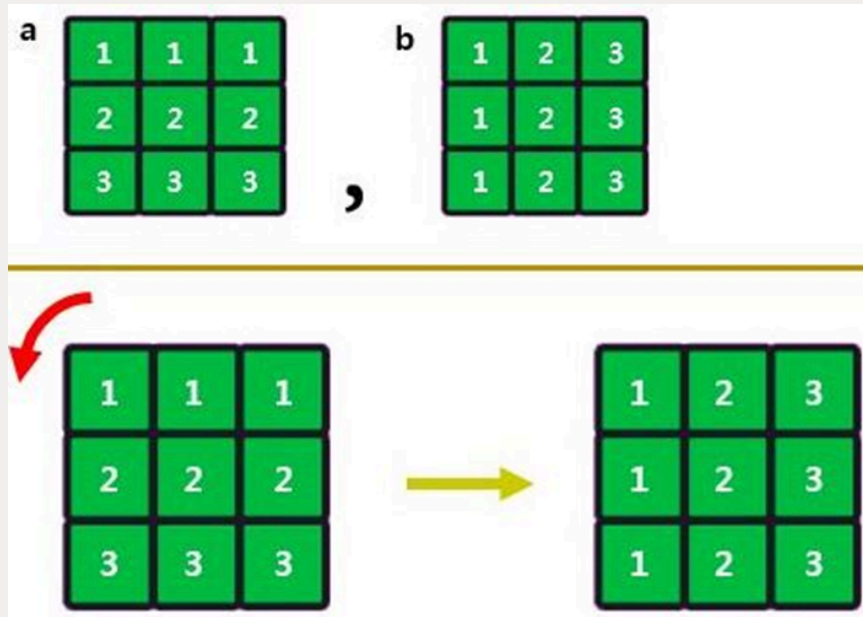
24



문제 3번

3×3으로 이루어진 a 배열과, b 배열을 입력받으세요

a의 네모블럭을 왼쪽으로 몇번 돌리면 b네모블럭과 같이 숫자가 배치 되는지
그 회전수를 출력 해주세요.



왼쪽으로 한번 돌리면 되므로 출력결과는 1 입니다.

입력 예제

```

1 1 1
2 2 2
3 3 3
1 2 3
1 2 3
1 2 3

```

출력 결과

1



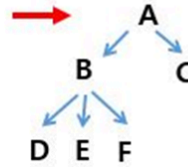
문제 4번

트리의 인접행렬을 입력받고

모든 경로를 출력해주세요. (노드는 A ~ F 까지 총 6개)

ex)

	A	B	C	D	E	F
A	0	1	1	0	0	0
B	0	0	0	1	1	1
C	0	0	0	0	0	0
D	0	0	0	0	0	0
E	0	0	0	0	0	0
F	0	0	0	0	0	0

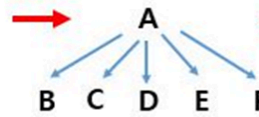


[출력결과]

ABD
ABE
ABF
AC

ex)

	A	B	C	D	E	F
A	0	1	1	1	1	1
B	0	0	0	0	0	0
C	0	0	0	0	0	0
D	0	0	0	0	0	0
E	0	0	0	0	0	0
F	0	0	0	0	0	0



[출력결과]

AB
AC
AD
AE
AF

입력 예제

011000
000111
000000
000000
000000
000000
000000

출력 결과

ABD
ABE
ABF
AC



문제 5번

n과 n개의 숫자를 입력 받으세요. ($1 \leq n \leq 5$)

입력 받은 숫자를 조합하여 가장 작은 세 자리수를 출력 해주세요.

ex)

[입력] [출력]

4

9 1 3 0 103

ex)

[입력] [출력]

5

0 0 0 0 5 500

입력 예제

4

9 1 3 0

출력 결과

103



문제 6

행과 열의 사이즈를 입력받아주세요.

그리고 행과 열 사이즈에 맞는 2차배열을 입력받아주세요.

이 중에서 가장 큰 숫자 값 3개와 그 값의 좌표를 출력 하세요.

1	5	2	7
1	5	1	6
3	3	2	4

<출력 결과>

큰값

7(0, 3)

6(1, 3)

5(0, 1)

입력 예제

3 4

1 5 2 7

1 5 1 6

3 3 2 4

출력 결과

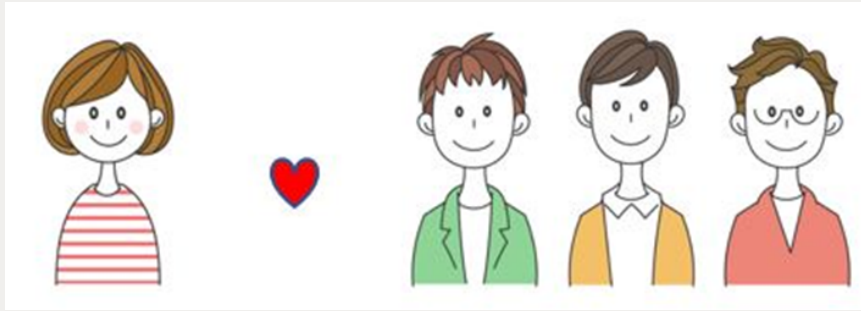
7(0,3)

6(1,3)

5(0,1)



문제 7번



세명의 이름을 입력 받으세요

영희는 순서대로 데이트를 할 상대를 선택해야 합니다.

월, 화, 수요일까지 총 세 번의 데이트를 하는데, 데이트 가능한 모든 경우를 출력 하세요.

(같은 사람을 지목해도 됩니다)

ex)

<입력>

bob jason tom

<출력>

bob bob bob

bob bob tom

bob bob jason

bob jason bob

bob jason jason

...

tom tom tom

입력 예제

bob jason tom

출력 결과

bob bob bob

bob bob jason

bob bob tom

bob jason bob

bob jason jason

bob jason tom

bob tom bob

bob tom jason

bob tom tom

jason bob bob

jason bob jason

jason bob tom

jason jason bob

jason jason jason

jason jason tom

jason tom bob

jason tom jason

jason tom tom

tom bob bob

tom bob jason

tom bob tom

tom jason bob

tom jason jason

tom jason tom

tom tom bob

tom tom jason
tom tom tom